

# SmartCart

## *Enterprise REST API Reference*

15 Controllers • 49 Endpoints • Spring Boot 3 + springdoc-openapi

**Project:** SmartCart – Full Stack E-Commerce Web Application

**Prepared by:** Darshan Takarkhede

**Base URL (Production):** [smartcart-backend-2muc.onrender.com](https://smartcart-backend-2muc.onrender.com)

**Interactive Docs (Swagger UI):** </swagger-ui/index.html>

**OpenAPI Spec:** </v3/api-docs>

**Source of Truth:** Backend source code + production database export

**Date:** August 2026

# Table of Contents

---

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>1. Overview</b>                                               | <b>3</b>  |
| 1.1 Purpose & Scope                                              | 3         |
| 1.2 Data Source & Methodology                                    | 3         |
| 1.3 Base URLs                                                    | 3         |
| 1.4 Authentication & Authorization (JWT)                         | 3         |
| 1.5 Standard Error Response Format                               | 4         |
| 1.6 Interactive Documentation                                    | 4         |
| 1.7 General Best Practices for Consumers                         | 4         |
| <b>2. Common Data Models</b>                                     | <b>6</b>  |
| <b>3. Authentication Endpoints</b>                               | <b>9</b>  |
| 3.1 Notes & Best Practices                                       | 10        |
| <b>4. Product &amp; Category Endpoints</b>                       | <b>11</b> |
| 4.1 Products                                                     | 11        |
| 4.2 Categories                                                   | 19        |
| 4.3 Notes & Best Practices                                       | 21        |
| <b>5. Cart, Wishlist &amp; Review Endpoints</b>                  | <b>22</b> |
| 5.1 Cart                                                         | 22        |
| 5.2 Wishlist                                                     | 24        |
| 5.3 Reviews                                                      | 26        |
| 5.4 Notes & Best Practices                                       | 27        |
| <b>6. Order, Payment &amp; Coupon Endpoints</b>                  | <b>28</b> |
| 6.1 Orders                                                       | 28        |
| 6.2 Notes & Best Practices — Orders                              | 31        |
| 6.3 Payments                                                     | 32        |
| 6.4 Coupons                                                      | 33        |
| 6.5 Notes & Best Practices — Payments & Coupons                  | 34        |
| <b>7. Profile, Dashboard, Reports &amp; Admin User Endpoints</b> | <b>35</b> |
| 7.1 Profile                                                      | 35        |
| 7.2 Dashboard                                                    | 36        |
| 7.3 Reports                                                      | 37        |
| 7.4 Admin User Management                                        | 39        |
| 7.5 Health Check                                                 | 41        |
| 7.6 Notes & Best Practices                                       | 41        |
| <b>Appendix: Quick-Reference Endpoint Index</b>                  | <b>42</b> |
| A.6 Database Cross-Reference                                     | 44        |

# 1. Overview

## 1.1 Purpose & Scope

This document is an enterprise-style reference for every REST endpoint exposed by the SmartCart backend, grouped by functional module: Authentication; Products & Categories; Cart, Wishlist & Reviews; Orders, Payments & Coupons; and Profile, Dashboard, Reports & Admin. Each endpoint specifies its HTTP method, URL, authorization requirement, parameters, request/response bodies, status codes, and a runnable sample request, so the document is usable both as a certification deliverable and as a working reference for integrating against the API.

## 1.2 Data Source & Methodology

This documentation is generated from two authoritative, primary sources rather than paraphrased from memory:

- The backend's actual Java source — every controller (15 files, 49 endpoint methods) and every request/response DTO — which is the exact code springdoc-openapi introspects to generate the live OpenAPI specification. Reading it directly guarantees this document matches the specification method-for-method, parameter-for-parameter.
- The production database export (smartcart\_db.sql), used to source realistic, real example values — actual product names and prices, actual category names, actual coupon codes, and actual order/payment records — rather than invented placeholder data.

*Note on methodology: an attempt was made to fetch the live specification directly from <https://smartcart-backend-2muc.onrender.com/v3/api-docs> for this revision, but the request was blocked by the host's robots.txt (automated-access disallowed). The endpoint remains fully usable interactively via a browser or a tool such as Postman — it simply cannot be auto-fetched by this tool. Since the specification is generated directly from the same controller/DTO source read for this document, the two are expected to match; cross-checking the live Swagger UI before submission is still recommended.*

## 1.3 Base URLs

| Environment         | Base URL                                                                                              |
|---------------------|-------------------------------------------------------------------------------------------------------|
| Local development   | <a href="http://localhost:8080">http://localhost:8080</a>                                             |
| Production (Render) | <a href="https://smartcart-backend-2muc.onrender.com">https://smartcart-backend-2muc.onrender.com</a> |

All endpoint paths and sample requests in this document are relative to one of these base URLs.

*The backend is hosted on Render's free tier, which spins down after a period of inactivity; the first request after idle time may take 30–50 seconds while the service restarts.*

## 1.4 Authentication & Authorization (JWT)

SmartCart uses stateless JWT bearer authentication with role-based access control. After a successful call to POST /api/auth/login, the response includes a signed token that must be attached to every subsequent request requiring authentication:

```
Authorization: Bearer <token>
```

Every endpoint in this document is labelled with one of three authorization levels:

- Public — no Authorization header required.
- Customer — requires a valid JWT for a user holding ROLE\_CUSTOMER (an ROLE\_ADMIN token also satisfies this, since admins can act on shared endpoints).
- Admin — requires a valid JWT for a user holding ROLE\_ADMIN; enforced server-side via Spring Security's @PreAuthorize on the controller method (or, for /api/admin/users/\*\*, at the class level).

Two seeded accounts exist in the production database and are used for the worked examples throughout this document:

| Email                  | Role          | Used in examples as  |
|------------------------|---------------|----------------------|
| admin@smartcart.com    | ROLE_ADMIN    | the Admin persona    |
| customer@smartcart.com | ROLE_CUSTOMER | the Customer persona |

## 1.5 Standard Error Response Format

Errors are handled centrally by a single @RestControllerAdvice (GlobalExceptionHandler) and returned as JSON in a consistent shape:

*Example error response*

```
{
  "message": "Descriptive error message"
}
```

| HTTP Status      | Meaning                                           | Example Trigger                                                     |
|------------------|---------------------------------------------------|---------------------------------------------------------------------|
| 400 Bad Request  | Validation failure or business-rule violation     | Duplicate email on register; empty cart at checkout; expired coupon |
| 401 Unauthorized | Missing/invalid/expired JWT, or wrong credentials | Login with wrong password; no bearer token supplied                 |
| 403 Forbidden    | Valid JWT, but role lacks permission              | Customer calling an Admin-only endpoint                             |
| 404 Not Found    | Referenced resource does not exist                | GET /api/products/{id} with an unknown ID                           |

## 1.6 Interactive Documentation

- Swagger UI: [smartcart-backend-2muc.onrender.com/swagger-ui/index.html](https://smartcart-backend-2muc.onrender.com/swagger-ui/index.html)
- Raw OpenAPI 3 specification (JSON): [smartcart-backend-2muc.onrender.com/v3/api-docs](https://smartcart-backend-2muc.onrender.com/v3/api-docs)

## 1.7 General Best Practices for Consumers

- Always send Content-Type: application/json on requests with a JSON body (multipart/form-data is the one exception, for image upload).
- Cache the JWT client-side and re-authenticate via `/api/auth/login` on a 401 rather than retrying the original request blindly.
- Treat all monetary fields (price, amount, totalAmount, etc.) as decimal values in Indian Rupees (₹); the API does not include a currency field, so this is a fixed, implicit convention.
- Prefer the paginated GET `/api/products/paged` endpoint over GET `/api/products` for large catalogues or infinite-scroll UIs.
- Treat 404 and 403 differently in client error handling: 404 means the resource genuinely does not exist, while 403 means it exists but the caller's role is not permitted — conflating them produces misleading end-user messages.

## 2. Common Data Models

Several endpoints return JPA entities directly rather than a trimmed DTO, so the same JSON shapes recur across multiple endpoints. They are documented once here, with real example values drawn from the production database, and referenced by name in the endpoint reference that follows.

### User

```
{
  "id": 2,
  "name": "Customer",
  "email": "customer@smartcart.com",
  "password": "$2a$10$M9Dp0EpYQhWVSyyCgleRj0...(BCrypt hash)",
  "role": "ROLE_CUSTOMER",
  "createdAt": "2026-07-16T14:32:52"
}
```

*Finding: the User entity has no @JsonIgnore on the password field. The value returned is a one-way BCrypt hash, not the plaintext password — but it is still present in the nested user object returned by cart, wishlist, review, and order endpoints, and directly in GET /api/admin/users. This is flagged as a hardening recommendation in Chapter 8 of the Project Report (add @JsonIgnore to User.password, or return a UserSummaryDTO instead of the entity).*

### Category

9 categories exist in production: Electronics, Clothing, Books, Footwear, Accessories, Toys, Grocery, Kitchen, Gaming.

```
{
  "id": 2,
  "name": "Electronics"
}
```

### Product

```
{
  "id": 4,
  "name": "MacBook Pro",
  "description": "Apple Laptop",
  "price": 149999,
  "stock": 13,
  "imageUrl": "mac.jpg",
  "category": {
    "id": 2,
    "name": "Electronics"
  }
}
```

*Finding: in the production database, imageUrl currently stores a bare filename (e.g. "mac.jpg") rather than a full Cloudinary secure URL. This differs from the Cloudinary-based description elsewhere in the project (Project Report, Chapter 4/14), which documents the intended/current-code behaviour of uploading to Cloudinary and storing the returned secure URL. This may indicate the exported dataset*

*predates a data migration, or that existing rows were seeded directly rather than through the /api/products/upload flow — worth reconciling before final submission.*

## Cart

```
{
  "id": 145,
  "user": "{ User }",
  "product": "{ Product }",
  "quantity": 1
}
```

## Wishlist

```
{
  "id": 65,
  "user": "{ User }",
  "product": "{ Product }"
}
```

## Review

```
{
  "id": 1,
  "user": "{ User }",
  "product": "{ Product }",
  "rating": 5,
  "comment": "Excellent product",
  "reviewDate": "2026-06-16T15:16:06"
}
```

## Coupon

*3 coupons exist in production: WELCOME10 (10%), NEWUSER20 (20%), FIRSTBUY15 (15%), all currently active.*

```
{
  "id": 1,
  "code": "WELCOME10",
  "discount": 10,
  "active": true
}
```

## Order & OrderItem

```
{
  "id": 43,
  "status": "PENDING",
  "paymentStatus": "SUCCESS",
  "couponCode": "WELCOME10",
  "originalAmount": 219998,
  "discountAmount": 21999.8,
  "totalAmount": 197998.2,
  "user": "{ User }",
  "orderDate": "2026-06-29T13:00:25",
}
```

```
"orderItems": [  
  {  
    "id": 57,  
    "product": "{ Product: MacBook Pro }",  
    "quantity": 1,  
    "price": 149999  
  },  
  {  
    "id": 58,  
    "product": "{ Product: Samsung S25 }",  
    "quantity": 1,  
    "price": 69999  
  }  
]  
}
```

*This is a real order from the production data: two items totalling ₹219,998, with the WELCOME10 coupon (10%) applying a ₹21,999.80 discount for a final total of ₹197,998.20 — the figures reconcile exactly, confirming the checkout discount logic.*

### Payment (PaymentResponseDTO)

```
{  
  "id": 20,  
  "amount": 149999,  
  "paymentMethod": "UPI",  
  "paymentStatus": "SUCCESS",  
  "paymentDate": "2026-06-26T16:10:01"  
}
```



## 3. Authentication Endpoints — /api/auth

### POST /api/auth/register

Registers a new customer account. The password is hashed with BCrypt before being persisted; the account is created with role ROLE\_CUSTOMER.

**Authorization Required:** Public

*Request Body (JSON)*

```
{
  "name": "Priya Verma",
  "email": "priya.verma@example.com",
  "password": "Secure@123"
}
```

*Sample Request (cURL)*

```
curl -X POST https://smartcart-backend-2muc.onrender.com/api/auth/register \
-H "Content-Type: application/json" \
-d '{"name":"Priya Verma","email":"priya.verma@example.com","password":"Secure@123"}'
```

*Sample Response (200 OK)*

```
{
  "message": "User registered successfully"
}
```

**HTTP Status Codes:**

| HTTP Status     | Meaning                                                                                                  |
|-----------------|----------------------------------------------------------------------------------------------------------|
| 200 OK          | Account created successfully                                                                             |
| 400 Bad Request | Email already registered, or validation failure (blank name, invalid email, password under 6 characters) |

### POST /api/auth/login

Authenticates a user and returns a signed JWT valid for subsequent requests.

**Authorization Required:** Public

*Request Body (JSON)*

```
{
  "email": "customer@smartcart.com",
  "password": "Test@123"
}
```

*Sample Request (cURL)*

```
curl -X POST https://smartcart-backend-2muc.onrender.com/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"customer@smartcart.com","password":"Test@123"}'
```

**Sample Response (200 OK)**

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "name": "Customer",
  "email": "customer@smartcart.com",
  "role": "ROLE_CUSTOMER"
}
```

**HTTP Status Codes:**

| HTTP Status      | Meaning                             |
|------------------|-------------------------------------|
| 200 OK           | Authenticated; JWT returned         |
| 401 Unauthorized | Incorrect password or unknown email |

**3.1 Notes & Best Practices**

- The registration endpoint does not auto-login; call `/api/auth/login` separately to obtain a token after registering.
- Store the returned token securely (e.g., in memory or an `HttpOnly`-equivalent storage strategy for SPAs); avoid storing long-lived tokens in `localStorage` where XSS could exfiltrate them.
- Role is server-assigned (`ROLE_CUSTOMER`) at registration — there is no client-supplied role field, which correctly prevents privilege escalation at signup.

## 4. Product & Category Endpoints

### 4.1 Products — /api/products

#### POST /api/products

Creates a new product.

**Authorization Required:** Admin

**Request Body (JSON)**

```
{
  "name": "Wireless Keyboard",
  "description": "Compact wireless keyboard",
  "price": 1999,
  "stock": 25,
  "imageUrl": "keyboard.jpg",
  "categoryId": 2
}
```

**Sample Request (cURL)**

```
curl -X POST https://smartcart-backend-2muc.onrender.com/api/products \
-H "Authorization: Bearer <admin_token>" \
-H "Content-Type: application/json" \
-d '{"name":"Wireless Keyboard","description":"Compact wireless
keyboard","price":1999.0,"stock":25,"imageUrl":"keyboard.jpg","categoryId":2}'
```

**Sample Response (200 OK)**

```
{
  "id": 18,
  "name": "Wireless Keyboard",
  "description": "Compact wireless keyboard",
  "price": 1999,
  "stock": 25,
  "imageUrl": "keyboard.jpg",
  "category": {
    "id": 2,
    "name": "Electronics"
  }
}
```

**HTTP Status Codes:**

| HTTP Status     | Meaning                  |
|-----------------|--------------------------|
| 200 OK          | Product created          |
| 403 Forbidden   | Caller is not ROLE_ADMIN |
| 400 Bad Request | Validation failure       |

## GET /api/products

Returns every product in the catalogue.

**Authorization Required:** Public

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/products
```

**Sample Response (200 OK)**

```
[
  {
    "id": 1,
    "name": "iPhone 16",
    "price": 79999,
    "stock": 17,
    "category": {
      "id": 2,
      "name": "Electronics"
    }
  },
  {
    "id": 4,
    "name": "MacBook Pro",
    "price": 149999,
    "stock": 13,
    "category": {
      "id": 2,
      "name": "Electronics"
    }
  },
  {
    "id": 14,
    "name": "Nike Sneakers",
    "price": 26999,
    "stock": 3,
    "category": {
      "id": 8,
      "name": "Footwear"
    }
  }
]
```

**HTTP Status Codes:**

| HTTP Status | Meaning                                           |
|-------------|---------------------------------------------------|
| 200 OK      | Array returned (empty array if no products exist) |

## GET /api/products/{id}

Returns a single product by its ID.

**Authorization Required:** Public

**Path Parameters:** id (Long) — product identifier, e.g. 4

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/products/4
```

**Sample Response (200 OK)**

```
{
  "id": 4,
  "name": "MacBook Pro",
  "description": "Apple Laptop",
  "price": 149999,
  "stock": 13,
  "category": {
    "id": 2,
    "name": "Electronics"
  }
}
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                      |
|---------------|------------------------------|
| 200 OK        | Product found                |
| 404 Not Found | No product with the given ID |

**PUT /api/products/{id}**

Updates an existing product's details.

**Authorization Required:** Admin

**Path Parameters:** id (Long) — product identifier

**Request Body (JSON)**

```
{
  "name": "MacBook Pro",
  "description": "Apple Laptop",
  "price": 144999,
  "stock": 13,
  "imageUrl": "mac.jpg",
  "categoryId": 2
}
```

**Sample Request (cURL)**

```
curl -X PUT https://smartcart-backend-2muc.onrender.com/api/products/4 \
-H "Authorization: Bearer <admin_token>" \
-H "Content-Type: application/json" \
-d '{"name":"MacBook Pro","description":"Apple Laptop","price":144999.0,"stock":13,"imageUrl":"mac.jpg","categoryId":2}'
```

**Sample Response (200 OK)**

```
{
  "id": 4,
  "name": "MacBook Pro",
  "price": 144999,
  "stock": 13,
  "category": {
    "id": 2,
    "name": "Electronics"
  }
}
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Product updated          |
| 403 Forbidden | Caller is not ROLE_ADMIN |
| 404 Not Found | Unknown product ID       |

**DELETE** /api/products/{id}

Deletes a product.

**Authorization Required:** Admin

**Path Parameters:** id (Long) — product identifier

**Sample Request (cURL)**

```
curl -X DELETE https://smartcart-backend-2muc.onrender.com/api/products/18 -H "Authorization: Bearer <admin_token>"
```

**Sample Response (200 OK)**

```
"Product Deleted"
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Product deleted          |
| 403 Forbidden | Caller is not ROLE_ADMIN |
| 404 Not Found | Unknown product ID       |

**Note:** Deleting a product that already appears in historical order\_items does not retroactively affect past orders, since OrderItem stores its own price/quantity snapshot rather than a live reference.

**GET** /api/products/category/{categoryId}

Returns all products belonging to a given category.

**Authorization Required:** Public

**Path Parameters:** categoryId (Long) — e.g. 8 (Footwear)

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/products/category/8
```

**Sample Response (200 OK)**

```
[
  {
    "id": 14,
    "name": "Nike Sneakers",
    "price": 26999,
    "category": {
      "id": 8,
      "name": "Footwear"
    }
  },
]
```

```
{
  "id": 16,
  "name": "Puma Sneakers",
  "price": 13799,
  "category": {
    "id": 8,
    "name": "Footwear"
  }
},
{
  "id": 17,
  "name": "Nike Mercurial",
  "price": 42999,
  "category": {
    "id": 8,
    "name": "Footwear"
  }
}
]
```

**HTTP Status Codes:**

| HTTP Status | Meaning                         |
|-------------|---------------------------------|
| 200 OK      | Array returned (possibly empty) |

**GET /api/products/search**

Keyword search across product name/description.

**Authorization Required:** Public

**Query Parameters:** keyword (String, required)

**Sample Request (cURL)**

```
curl "https://smartcart-backend-2muc.onrender.com/api/products/search?keyword=Nike"
```

**Sample Response (200 OK)**

```
[
  {
    "id": 14,
    "name": "Nike Sneakers",
    "price": 26999
  },
  {
    "id": 17,
    "name": "Nike Mercurial",
    "price": 42999
  }
]
```

**HTTP Status Codes:**

| HTTP Status     | Meaning                         |
|-----------------|---------------------------------|
| 200 OK          | Array returned (possibly empty) |
| 400 Bad Request | keyword parameter missing       |

## GET /api/products/sort/price-asc

Returns all products ordered by price, lowest first.

**Authorization Required:** Public

### Sample Request (cURL)

```
curl https://smartcart-backend-2muc.onrender.com/api/products/sort/price-asc
```

### Sample Response (200 OK)

```
[
  {
    "id": 13,
    "name": "The Photographer's Handbook",
    "price": 1699
  },
  {
    "id": 12,
    "name": "Hoodie",
    "price": 3499
  }
]
```

### HTTP Status Codes:

| HTTP Status | Meaning                         |
|-------------|---------------------------------|
| 200 OK      | Array returned, ascending price |

## GET /api/products/sort/price-desc

Returns all products ordered by price, highest first.

**Authorization Required:** Public

### Sample Request (cURL)

```
curl https://smartcart-backend-2muc.onrender.com/api/products/sort/price-desc
```

### Sample Response (200 OK)

```
[
  {
    "id": 4,
    "name": "MacBook Pro",
    "price": 149999
  },
  {
    "id": 1,
    "name": "iPhone 16",
    "price": 79999
  }
]
```

### HTTP Status Codes:

| HTTP Status | Meaning                          |
|-------------|----------------------------------|
| 200 OK      | Array returned, descending price |



## GET /api/products/paged

Returns a paginated slice of the product catalogue using Spring Data's Page abstraction.

**Authorization Required:** Public

**Query Parameters:** page (int, default 0), size (int, default 5)

### Sample Request (cURL)

```
curl "https://smartcart-backend-2muc.onrender.com/api/products/paged?page=0&size=5"
```

### Sample Response (200 OK)

```
{
  "content": [
    {
      "id": 1,
      "name": "iPhone 16",
      "price": 79999
    }
  ],
  "totalElements": 8,
  "totalPages": 2,
  "number": 0,
  "size": 5
}
```

### HTTP Status Codes:

| HTTP Status | Meaning                |
|-------------|------------------------|
| 200 OK      | Page<Product> returned |

## PUT /api/products/{id}/stock

Updates only the stock quantity for a product (used by the low-stock workflow).

**Authorization Required:** Admin

**Path Parameters:** id (Long) — product identifier

### Request Body (JSON)

```
{
  "stock": 3
}
```

### Sample Request (cURL)

```
curl -X PUT https://smartcart-backend-2muc.onrender.com/api/products/14/stock \
-H "Authorization: Bearer <admin_token>" \
-H "Content-Type: application/json" \
-d '{"stock":3}'
```

### Sample Response (200 OK)

```
{
  "id": 14,
  "name": "Nike Sneakers",
  "stock": 3
}
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Stock updated            |
| 403 Forbidden | Caller is not ROLE_ADMIN |

**GET** /api/products/low-stock

Returns products whose stock has fallen below the configured low-stock threshold.

**Authorization Required:** Admin

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/products/low-stock -H "Authorization: Bearer <admin_token>"
```

**Sample Response (200 OK)**

```
[
  {
    "id": 14,
    "name": "Nike Sneakers",
    "stock": 3
  },
  {
    "id": 4,
    "name": "MacBook Pro",
    "stock": 13
  }
]
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                         |
|---------------|---------------------------------|
| 200 OK        | Array returned (possibly empty) |
| 403 Forbidden | Caller is not ROLE_ADMIN        |

**POST** /api/products/upload

Uploads a product image directly to Cloudinary and returns the resulting secure URL. Uses multipart/form-data rather than JSON.

**Authorization Required:** Admin

**Request Body:** multipart/form-data; field name: file (the image file)

**Sample Request (cURL)**

```
curl -X POST https://smartcart-backend-2muc.onrender.com/api/products/upload \
-H "Authorization: Bearer <admin_token>" \
-F "file=@keyboard.jpg"
```

**Sample Response (200 OK)**

```
"https://res.cloudinary.com/smartcart/image/upload/v123456789/keyboard.jpg"
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                               |
|---------------|---------------------------------------|
| 200 OK        | Upload succeeded; secure URL returned |
| 403 Forbidden | Caller is not ROLE_ADMIN              |

## 4.2 Categories — /api/categories

### POST /api/categories

Creates a new category.

**Authorization Required:** Admin

*Request Body (JSON)*

```
{
  "name": "Home Appliances"
}
```

*Sample Request (cURL)*

```
curl -X POST https://smartcart-backend-2muc.onrender.com/api/categories \
-H "Authorization: Bearer <admin_token>" \
-H "Content-Type: application/json" \
-d '{"name":"Home Appliances"}
```

*Sample Response (200 OK)*

```
{
  "id": 14,
  "name": "Home Appliances"
}
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Category created         |
| 403 Forbidden | Caller is not ROLE_ADMIN |

### GET /api/categories

Returns every category.

**Authorization Required:** Public

*Sample Request (cURL)*

```
curl https://smartcart-backend-2muc.onrender.com/api/categories
```

**Sample Response (200 OK)**

```
[
  {
    "id": 2,
    "name": "Electronics"
  },
  {
    "id": 6,
    "name": "Clothing"
  },
  {
    "id": 7,
    "name": "Books"
  },
  {
    "id": 8,
    "name": "Footwear"
  }
]
```

**HTTP Status Codes:**

| HTTP Status | Meaning        |
|-------------|----------------|
| 200 OK      | Array returned |

**PUT /api/categories/{id}**

Updates a category's name.

**Authorization Required:** Admin

**Path Parameters:** id (Long) — category identifier

**Request Body (JSON)**

```
{
  "name": "Consumer Electronics"
}
```

**Sample Request (cURL)**

```
curl -X PUT https://smartcart-backend-2muc.onrender.com/api/categories/2 \
-H "Authorization: Bearer <admin_token>" \
-H "Content-Type: application/json" \
-d '{"name":"Consumer Electronics"}
```

**Sample Response (200 OK)**

```
{
  "id": 2,
  "name": "Consumer Electronics"
}
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Category updated         |
| 403 Forbidden | Caller is not ROLE_ADMIN |
| 404 Not Found | Unknown category ID      |

**DELETE** /api/categories/{id}

Deletes a category.

**Authorization Required:** Admin

**Path Parameters:** id (Long) — category identifier

**Sample Request (cURL)**

```
curl -X DELETE https://smartcart-backend-2muc.onrender.com/api/categories/14 -H
"Authorization: Bearer <admin_token>"
```

**Sample Response (200 OK)**

```
"Category Deleted"
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Category deleted         |
| 403 Forbidden | Caller is not ROLE_ADMIN |
| 404 Not Found | Unknown category ID      |

**Note:** The current implementation does not check whether products still reference the category before deleting; deleting a category that products still point to is a referential-integrity edge case worth guarding against (see Chapter 15, Future Enhancements).

## 4.3 Notes & Best Practices

- categoryId in ProductDTO must reference an existing category; there is no cascading auto-create of categories from the product endpoints.
- Price and stock accept any Double/Integer value at the DTO level — there is currently no server-side @Min(0) constraint, so negative values are not rejected by validation alone (see Chapter 8 of the Test Cases document, TC-017).
- /api/products/upload only returns the Cloudinary URL; it does not create or update a product. Call it first, then pass the returned URL as imageUrl in a subsequent POST/PUT /api/products call.
- Prefer /api/products/search and /api/products/category/{categoryId} over client-side filtering of the full GET /api/products result for anything beyond a trivially small catalogue.

## 5. Cart, Wishlist & Review Endpoints

### 5.1 Cart — /api/cart

All cart endpoints require a Customer JWT. The current user is resolved server-side from the token, so a customer can only ever see or modify their own cart. Examples below use the seeded customer@smartcart.com account, whose real production cart contains a Nike Sneakers, a book, and a MacBook Pro line item.

#### POST /api/cart

Adds a product to the authenticated user's cart.

**Authorization Required:** Customer

**Request Body (JSON)**

```
{
  "productId": 14,
  "quantity": 1
}
```

**Sample Request (cURL)**

```
curl -X POST https://smartcart-backend-2muc.onrender.com/api/cart \
-H "Authorization: Bearer <customer_token>" \
-H "Content-Type: application/json" \
-d '{"productId":14,"quantity":1}'
```

**Sample Response (200 OK)**

```
{
  "id": 143,
  "user": "{ User }",
  "product": {
    "id": 14,
    "name": "Nike Sneakers",
    "price": 26999
  },
  "quantity": 1
}
```

**HTTP Status Codes:**

| HTTP Status      | Meaning               |
|------------------|-----------------------|
| 200 OK           | Item added            |
| 401 Unauthorized | Missing/invalid token |

## GET /api/cart

Returns the authenticated user's current cart items.

**Authorization Required:** Customer

### Sample Request (cURL)

```
curl https://smartcart-backend-2muc.onrender.com/api/cart -H "Authorization: Bearer <customer_token>"
```

### Sample Response (200 OK)

```
[
  {
    "id": 143,
    "product": {
      "id": 14,
      "name": "Nike Sneakers",
      "price": 26999
    },
    "quantity": 1
  },
  {
    "id": 144,
    "product": {
      "id": 13,
      "name": "The Photographer's Handbook",
      "price": 1699
    },
    "quantity": 1
  },
  {
    "id": 145,
    "product": {
      "id": 4,
      "name": "MacBook Pro",
      "price": 149999
    },
    "quantity": 1
  }
]
```

### HTTP Status Codes:

| HTTP Status | Meaning                         |
|-------------|---------------------------------|
| 200 OK      | Array returned (possibly empty) |

## DELETE /api/cart/{cartId}

Removes a single item from the cart.

**Authorization Required:** Customer

**Path Parameters:** cartId (Long) — cart item identifier, e.g. 144

### Sample Request (cURL)

```
curl -X DELETE https://smartcart-backend-2muc.onrender.com/api/cart/144 -H "Authorization: Bearer <customer_token>"
```

**Sample Response (200 OK)**

```
"Item Removed"
```

**HTTP Status Codes:**

| HTTP Status   | Meaning              |
|---------------|----------------------|
| 200 OK        | Item removed         |
| 404 Not Found | Unknown cart item ID |

**PUT /api/cart/{cartId}**

Updates the quantity of an existing cart item.

**Authorization Required:** Customer

**Path Parameters:** cartId (Long) — cart item identifier

**Query Parameters:** quantity (Integer, required)

**Sample Request (cURL)**

```
curl -X PUT "https://smartcart-backend-2muc.onrender.com/api/cart/145?quantity=2" -H  
"Authorization: Bearer <customer_token>"
```

**Sample Response (200 OK)**

```
{  
  "id": 145,  
  "product": {  
    "id": 4,  
    "name": "MacBook Pro",  
    "price": 149999  
  },  
  "quantity": 2  
}
```

**HTTP Status Codes:**

| HTTP Status   | Meaning              |
|---------------|----------------------|
| 200 OK        | Quantity updated     |
| 404 Not Found | Unknown cart item ID |

**5.2 Wishlist — /api/wishlist****POST /api/wishlist**

Adds a product to the authenticated user's wishlist.

**Authorization Required:** Customer

**Request Body (JSON)**

```
{  
  "productId": 4  
}
```



**Sample Request (cURL)**

```
curl -X POST https://smartcart-backend-2muc.onrender.com/api/wishlist \
-H "Authorization: Bearer <customer_token>" \
-H "Content-Type: application/json" \
-d '{"productId":4}'
```

**Sample Response (200 OK)**

```
{
  "id": 65,
  "product": {
    "id": 4,
    "name": "MacBook Pro",
    "price": 149999
  }
}
```

**HTTP Status Codes:**

| HTTP Status | Meaning    |
|-------------|------------|
| 200 OK      | Item added |

**GET /api/wishlist**

Returns the authenticated user's wishlist.

**Authorization Required:** Customer

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/wishlist -H "Authorization: Bearer <customer_token>"
```

**Sample Response (200 OK)**

```
[
  {
    "id": 64,
    "product": {
      "id": 14,
      "name": "Nike Sneakers",
      "price": 26999
    }
  },
  {
    "id": 65,
    "product": {
      "id": 4,
      "name": "MacBook Pro",
      "price": 149999
    }
  },
  {
    "id": 66,
    "product": {
      "id": 13,
      "name": "The Photographer's Handbook",
      "price": 1699
    }
  }
]
```

**HTTP Status Codes:**

| HTTP Status | Meaning                         |
|-------------|---------------------------------|
| 200 OK      | Array returned (possibly empty) |

**DELETE** `/api/wishlist/{id}`

Removes an item from the wishlist.

**Authorization Required:** Customer

**Path Parameters:** id (Long) — wishlist item identifier

**Sample Request (cURL)**

```
curl -X DELETE https://smartcart-backend-2muc.onrender.com/api/wishlist/66 -H "Authorization: Bearer <customer_token>"
```

**Sample Response (200 OK)**

```
"Wishlist Item Removed"
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Item removed             |
| 404 Not Found | Unknown wishlist item ID |

## 5.3 Reviews — `/api/reviews`

**POST** `/api/reviews`

Submits a rating and comment for a product.

**Authorization Required:** Customer

**Request Body (JSON)**

```
{
  "productId": 4,
  "rating": 4,
  "comment": "best performance"
}
```

**Sample Request (cURL)**

```
curl -X POST https://smartcart-backend-2muc.onrender.com/api/reviews \
-H "Authorization: Bearer <customer_token>" \
-H "Content-Type: application/json" \
-d '{"productId":4,"rating":4,"comment":"best performance"}
```

**Sample Response (200 OK)**

```
{
  "id": 17,
  "product": {
    "id": 4,
    "name": "MacBook Pro"
  },
  "rating": 4,
  "comment": "best performance",
  "reviewDate": "2026-06-26T16:09:32"
}
```

**HTTP Status Codes:**

| HTTP Status | Meaning      |
|-------------|--------------|
| 200 OK      | Review saved |

**GET /api/reviews/{productId}**

Returns all reviews for a given product.

**Authorization Required:** Public

**Path Parameters:** productId (Long) — e.g. 4 (MacBook Pro)

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/reviews/4
```

**Sample Response (200 OK)**

```
[
  {
    "id": 17,
    "rating": 4,
    "comment": "best performance",
    "reviewDate": "2026-06-26T16:09:32"
  }
]
```

**HTTP Status Codes:**

| HTTP Status | Meaning                         |
|-------------|---------------------------------|
| 200 OK      | Array returned (possibly empty) |

## 5.4 Notes & Best Practices

- There is no uniqueness constraint preventing the same user from reviewing the same product multiple times — the production data shows one user with 13 reviews on a single product (id 1). Consider a unique (user\_id, product\_id) constraint if one-review-per-purchase is the intended behaviour.
- Wishlist and Cart are independent: adding a product to one does not add or remove it from the other.
- rating is stored as a plain int with no server-side @Min(1)/@Max(5) constraint currently enforced — client-side validation should not be treated as sufficient on its own.

## 6. Order, Payment & Coupon Endpoints

### 6.1 Orders — /api/orders

#### POST /api/orders/checkout

Converts the authenticated user's current cart into an order. If a coupon code is supplied and is active, its discount is applied to the subtotal. Runs inside a single @Transactional boundary covering stock validation, order/order-item creation, and the confirmation email.

**Authorization Required:** Customer

**Query Parameters:** couponCode (String, optional) — e.g. ?couponCode=WELCOME10

##### Sample Request (cURL)

```
curl -X POST "https://smartcart-backend-2muc.onrender.com/api/orders/checkout?couponCode=WELCOME10" \
-H "Authorization: Bearer <customer_token>"
```

##### Sample Response (200 OK)

```
{
  "id": 43,
  "status": "PENDING",
  "paymentStatus": "SUCCESS",
  "couponCode": "WELCOME10",
  "originalAmount": 219998,
  "discountAmount": 21999.8,
  "totalAmount": 197998.2,
  "orderDate": "2026-06-29T13:00:25",
  "orderItems": [
    {
      "id": 57,
      "product": {
        "id": 4,
        "name": "MacBook Pro"
      },
      "quantity": 1,
      "price": 149999
    },
    {
      "id": 58,
      "product": {
        "id": 3,
        "name": "Samsung S25"
      },
      "quantity": 1,
      "price": 69999
    }
  ]
}
```

**HTTP Status Codes:**

| HTTP Status     | Meaning                                |
|-----------------|----------------------------------------|
| 200 OK          | Order created                          |
| 400 Bad Request | "Cart is Empty"                        |
| 400 Bad Request | "Coupon Expired" / invalid coupon code |

**Note:** This example reproduces an actual production order (ID 43): ₹219,998 subtotal, WELCOME10's 10% discount, ₹197,998.20 final total — the arithmetic in the live database matches the documented discount logic exactly.

**GET /api/orders**

Returns the authenticated customer's order history.

**Authorization Required:** Customer

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/orders -H "Authorization: Bearer <customer_token>"
```

**Sample Response (200 OK)**

```
[
  {
    "id": 3,
    "status": "DELIVERED",
    "totalAmount": 159998,
    "orderDate": "2026-06-16T14:45:37"
  },
  {
    "id": 22,
    "status": "SHIPPED",
    "totalAmount": 69999,
    "orderDate": "2026-06-23T16:24:58"
  },
  {
    "id": 33,
    "status": "SHIPPED",
    "totalAmount": 149999,
    "orderDate": "2026-06-26T16:09:56"
  }
]
```

**HTTP Status Codes:**

| HTTP Status | Meaning                         |
|-------------|---------------------------------|
| 200 OK      | Array returned (possibly empty) |

**GET /api/orders/admin**

Returns every order placed on the platform, regardless of customer.

**Authorization Required:** Admin

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/orders/admin -H "Authorization: Bearer <admin_token>"
```

**Sample Response (200 OK)**

```
[
  {
    "id": 33,
    "user": {
      "id": 2,
      "email": "customer@smartcart.com"
    },
    "status": "SHIPPED",
    "totalAmount": 149999
  },
  {
    "id": 43,
    "user": {
      "id": 4,
      "email": "customer01@smartcart.com"
    },
    "status": "PENDING",
    "totalAmount": 197998.2
  }
]
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Array returned           |
| 403 Forbidden | Caller is not ROLE_ADMIN |

**PUT /api/orders/{id}/status**

Updates the status of an order (e.g., progressing it through PENDING → SHIPPED → DELIVERED).

**Authorization Required:** Admin

**Path Parameters:** id (Long) — order identifier, e.g. 71

**Request Body (JSON)**

```
{
  "status": "SHIPPED"
}
```

**Sample Request (cURL)**

```
curl -X PUT https://smartcart-backend-2muc.onrender.com/api/orders/71/status \
-H "Authorization: Bearer <admin_token>" \
-H "Content-Type: application/json" \
-d '{"status": "SHIPPED"}'
```

**Sample Response (200 OK)**

```
{
```

```
{
  "id": 71,
  "status": "SHIPPED"
}
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Status updated           |
| 403 Forbidden | Caller is not ROLE_ADMIN |

**PUT** `/api/orders/cancel/{id}`

Cancels an order that is still eligible for cancellation.

**Authorization Required:** Customer

**Path Parameters:** id (Long) — order identifier, e.g. 7

**Sample Request (cURL)**

```
curl -X PUT https://smartcart-backend-2muc.onrender.com/api/orders/cancel/7 -H
"Authorization: Bearer <customer_token>"
```

**Sample Response (200 OK)**

```
{
  "id": 7,
  "status": "CANCELLED"
}
```

**HTTP Status Codes:**

| HTTP Status     | Meaning                                      |
|-----------------|----------------------------------------------|
| 200 OK          | Order cancelled                              |
| 400 Bad Request | Order is no longer eligible for cancellation |

**GET** `/api/orders/invoice/{orderId}`

Generates and streams a PDF invoice for the given order using iText7.

**Authorization Required:** Customer

**Path Parameters:** orderId (Long) — order identifier, e.g. 33

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/orders/invoice/33 -H "Authorization:
Bearer <customer_token>" -o invoice_33.pdf
```

**Sample Response (200 OK)**

```
"Binary PDF stream (Content-Type: application/pdf, Content-Disposition: attachment; filename=invoice_33.pdf)"
```

**HTTP Status Codes:**

| HTTP Status   | Meaning             |
|---------------|---------------------|
| 200 OK        | PDF stream returned |
| 404 Not Found | Unknown order ID    |

## 6.2 Notes & Best Practices — Orders

- Checkout is atomic: if stock validation or coupon application fails partway through, the entire order creation rolls back — clients should treat a non-200 response from checkout as “nothing was created” and safe to retry.
- couponCode is matched case-sensitively against the stored code in the current implementation; sending "welcome10" instead of "WELCOME10" will not match unless the client uppercases it first (several historical rows in production show lowercase codes stored as-is, e.g. order 45 stores 'welcome10').
- originalAmount, discountAmount and couponCode are null on orders placed without a coupon — clients should null-check before displaying a discount line.

## 6.3 Payments — /api/payments

Payment processing is a simulated gateway for demonstration purposes (see Chapter 14 of the Project Report): each attempt is recorded with an 85% random success probability rather than being routed to a live payment processor. Every payment record in the production data uses "UPI" as the payment method.

### POST /api/payments

Processes a (simulated) payment attempt for an order.

**Authorization Required:** Customer

**Request Body (JSON)**

```
{
  "orderId": 71,
  "paymentMethod": "UPI"
}
```

**Sample Request (cURL)**

```
curl -X POST https://smartcart-backend-2muc.onrender.com/api/payments \
-H "Authorization: Bearer <customer_token>" \
-H "Content-Type: application/json" \
-d '{"orderId":71,"paymentMethod":"UPI"}
```



**Sample Response (200 OK)**

```
{
  "id": 56,
  "amount": 26999,
  "paymentMethod": "UPI",
  "paymentStatus": "SUCCESS",
  "paymentDate": "2026-07-15T14:37:29"
}
```

**HTTP Status Codes:**

| HTTP Status | Meaning                                      |
|-------------|----------------------------------------------|
| 200 OK      | Payment attempt recorded (SUCCESS or FAILED) |

**GET /api/payments**

Returns recorded payment attempts.

**Authorization Required:** Customer

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/payments -H "Authorization: Bearer <customer_token>"
```

**Sample Response (200 OK)**

```
[
  {
    "id": 20,
    "amount": 149999,
    "paymentMethod": "UPI",
    "paymentStatus": "SUCCESS"
  },
  {
    "id": 56,
    "amount": 26999,
    "paymentMethod": "UPI",
    "paymentStatus": "SUCCESS"
  }
]
```

**HTTP Status Codes:**

| HTTP Status | Meaning        |
|-------------|----------------|
| 200 OK      | Array returned |

## 6.4 Coupons — /api/coupons

### POST /api/coupons

Creates a new discount coupon.

**Authorization Required:** Admin

**Request Body (JSON)**

```
{
  "code": "FESTIVE25",
  "discount": 25,
  "active": true
}
```

**Sample Request (cURL)**

```
curl -X POST https://smartcart-backend-2muc.onrender.com/api/coupons \
-H "Authorization: Bearer <admin_token>" \
-H "Content-Type: application/json" \
-d '{"code":"FESTIVE25","discount":25.0,"active":true}'
```

**Sample Response (200 OK)**

```
{
  "id": 4,
  "code": "FESTIVE25",
  "discount": 25,
  "active": true
}
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Coupon created           |
| 403 Forbidden | Caller is not ROLE_ADMIN |

### GET /api/coupons/validate/{code}

Validates a coupon code and returns its discount percentage if active.

**Authorization Required:** Public

**Path Parameters:** code (String) — e.g. WELCOME10

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/coupons/validate/WELCOME10
```

**Sample Response (200 OK)**

```
"10.0"
```

**HTTP Status Codes:**

| HTTP Status | Meaning                              |
|-------------|--------------------------------------|
| 200 OK      | Discount percentage returned         |
| 400/404     | Coupon does not exist or is inactive |

## 6.5 Notes & Best Practices — Payments & Coupons

- Because payment success is randomised (~85%), automated tests against this endpoint should assert on the shape of the response and on paymentStatus being one of {SUCCESS, FAILED}, not on a specific outcome.
- There is no dedicated “retry payment” endpoint; a failed payment is retried by calling POST /api/payments again with the same orderId.
- Coupon codes have a database-level UNIQUE constraint (see smartcart\_db.sql, UKeptl0kkm9yf2of2lnx6c1oy9b on coupons.code), so creating a duplicate code will fail at the database layer.

## 7. Profile, Dashboard, Reports & Admin User Endpoints

### 7.1 Profile — /api/profile

#### **PUT** /api/profile

Updates the authenticated user's profile information.

**Authorization Required:** Customer/Admin

**Request Body (JSON)**

```
{
  "name": "Customer",
  "email": "customer@smartcart.com"
}
```

**Sample Request (cURL)**

```
curl -X PUT https://smartcart-backend-2muc.onrender.com/api/profile \
-H "Authorization: Bearer <customer_token>" \
-H "Content-Type: application/json" \
-d '{"name": "Customer", "email": "customer@smartcart.com"}'
```

**Sample Response (200 OK)**

```
{
  "name": "Customer",
  "email": "customer@smartcart.com"
}
```

**HTTP Status Codes:**

| HTTP Status | Meaning         |
|-------------|-----------------|
| 200 OK      | Profile updated |

#### **PUT** /api/profile/password

Changes the authenticated user's password after verifying the current password.

**Authorization Required:** Customer/Admin

**Request Body (JSON)**

```
{
  "currentPassword": "Test@123",
  "newPassword": "NewSecure@456"
}
```

**Sample Request (cURL)**

```
curl -X PUT https://smartcart-backend-2muc.onrender.com/api/profile/password \
-H "Authorization: Bearer <customer_token>" \
-H "Content-Type: application/json" \
-d '{"currentPassword": "Test@123", "newPassword": "NewSecure@456"}'
```

**Sample Response (200 OK)**

```
{
  "message": "Password updated successfully"
}
```

**HTTP Status Codes:**

| HTTP Status     | Meaning                         |
|-----------------|---------------------------------|
| 200 OK          | Password changed                |
| 400 Bad Request | Current password does not match |

**GET /api/profile/stats**

Returns summary statistics for the authenticated user's account.

**Authorization Required:** Customer/Admin

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/profile/stats -H "Authorization: Bearer <customer_token>"
```

**Sample Response (200 OK)**

```
{
  "totalOrders": 47,
  "wishlistItems": 3,
  "cartItems": 3,
  "memberSince": "Jul 2026"
}
```

**HTTP Status Codes:**

| HTTP Status | Meaning             |
|-------------|---------------------|
| 200 OK      | Statistics returned |

**Note:** Figure shown (totalOrders: 47) reflects that customer@smartcart.com (user id 2) is the primary test account in the production data, with more orders attributed to it than any other seeded user.

**7.2 Dashboard — /api/dashboard****GET /api/dashboard**

Returns aggregated metrics for the admin dashboard in a single call.

**Authorization Required:** Admin

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/dashboard -H "Authorization: Bearer <admin_token>"
```

**Sample Response (200 OK)**

```
{
  "totalUsers": 5,
  "totalProducts": 8,
  "totalOrders": 71,
  "totalRevenue": 10924393.5,
  "pendingOrders": 19,
  "shippedOrders": 25,
  "deliveredOrders": 18,
  "cancelledOrders": 9,
  "recentOrders": [
    "{ Order }"
  ]
}
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Metrics returned         |
| 403 Forbidden | Caller is not ROLE_ADMIN |

**Note:** All figures above are computed directly from the production database export (5 users, 8 products, 71 orders totalling ₹10,924,393.50) rather than invented, so they can be cross-checked against `smartcart_db.sql` directly.

## 7.3 Reports — /api/reports

### GET /api/reports/sales

Returns overall sales figures.

**Authorization Required:** Admin

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/reports/sales -H "Authorization: Bearer <admin_token>"
```

**Sample Response (200 OK)**

```
{
  "totalOrders": 71,
  "totalRevenue": 10924393.5,
  "averageOrderValue": 153864.7,
  "highestOrder": 899989
}
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Report returned          |
| 403 Forbidden | Caller is not ROLE_ADMIN |

## GET /api/reports/monthly-revenue

Returns a month-by-month revenue series for charting.

**Authorization Required:** Admin

### Sample Request (cURL)

```
curl https://smartcart-backend-2muc.onrender.com/api/reports/monthly-revenue -H
"Authorization: Bearer <admin_token>"
```

### Sample Response (200 OK)

```
[
  {
    "month": "Jun 2026",
    "revenue": 7842000
  },
  {
    "month": "Jul 2026",
    "revenue": 3082393.5
  }
]
```

### HTTP Status Codes:

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Array returned           |
| 403 Forbidden | Caller is not ROLE_ADMIN |

**Note:** Order dates in the production data span June–July 2026 only, so the live series currently has two data points.

## GET /api/reports/product-performance

Returns per-product sales performance.

**Authorization Required:** Admin

### Sample Request (cURL)

```
curl https://smartcart-backend-2muc.onrender.com/api/reports/product-performance -H
"Authorization: Bearer <admin_token>"
```

### Sample Response (200 OK)

```
[
  {
    "productName": "Samsung S25",
    "category": "Electronics",
    "unitsSold": 61,
    "revenue": 4269939,
    "averagePrice": 69999,
    "stockRemaining": 17
  },
  {
    "productName": "MacBook Pro",
    "category": "Electronics",
    "unitsSold": 22,
    "revenue": 3299978,
    "averagePrice": 149999,
    "stockRemaining": 13
  }
]
```

```
    },
    {
      "productName": "iPhone 16",
      "category": "Electronics",
      "unitsSold": 40,
      "revenue": 3199960,
      "averagePrice": 79999,
      "stockRemaining": 17
    }
  ]
```

#### HTTP Status Codes:

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Array returned           |
| 403 Forbidden | Caller is not ROLE_ADMIN |

**Note:** Figures computed directly from `order_items` in the production export. Samsung S25 leads on units sold (61) despite MacBook Pro having a higher unit price, and both outsell the flagship iPhone 16 listing in aggregate revenue terms.

### 7.3.1 Notes & Best Practices — Dashboard & Reports

- totalRevenue currently sums totalAmount across all orders regardless of paymentStatus — 9 of the 71 production orders have paymentStatus = FAILED, so this figure includes revenue that was never actually collected. Confirm whether this is intended or whether FAILED-payment orders should be excluded before presenting this figure to business stakeholders.
- /api/reports/monthly-revenue and /api/dashboard both derive from the same underlying order data but are computed independently — keep them consistent if the aggregation logic changes in one place.

## 7.4 Admin User Management — /api/admin/users

Every endpoint under this path is class-level protected with `@PreAuthorize("hasRole('ADMIN')")`.

### GET /api/admin/users

Returns every registered user.

**Authorization Required:** Admin

#### Sample Request (cURL)

```
curl https://smartcart-backend-2muc.onrender.com/api/admin/users -H "Authorization: Bearer <admin_token>"
```

#### Sample Response (200 OK)

```
[
  {
    "id": 1,
    "name": "Admin",
    "email": "admin@smartcart.com",
    "role": "ROLE_ADMIN"
  },
  {
```



```
[{"id": 2,
  "name": "Customer",
  "email": "customer@smartcart.com",
  "role": "ROLE_CUSTOMER"},
{"id": 4,
  "name": "Customer 01",
  "email": "customer01@smartcart.com",
  "role": "ROLE_CUSTOMER"}]
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Array returned           |
| 403 Forbidden | Caller is not ROLE_ADMIN |

**Note:** As documented in Chapter 2, each user object in this array includes the BCrypt password hash — see the *User model finding* for the recommended fix.

**DELETE** `/api/admin/users/{id}`

Deletes a user account.

**Authorization Required:** Admin

**Path Parameters:** id (Long) — user identifier

**Sample Request (cURL)**

```
curl -X DELETE https://smartcart-backend-2muc.onrender.com/api/admin/users/7 -H
"Authorization: Bearer <admin_token>"
```

**Sample Response (200 OK)**

```
"User deleted successfully"
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | User deleted             |
| 403 Forbidden | Caller is not ROLE_ADMIN |

**Note:** Deleting a user does not cascade-delete their historical orders in the current schema (`orders.user_id` has no `ON DELETE CASCADE` in `smartcart_db.sql`) — deleting a user with existing orders will fail at the database foreign-key constraint unless application logic handles it first.

**PUT** `/api/admin/users/{id}/role`

Changes a user's role.

**Authorization Required:** Admin

**Path Parameters:** id (Long) — user identifier

**Query Parameters:** role (String, required) — e.g. `?role=ROLE_ADMIN`

**Sample Request (cURL)**

```
curl -X PUT "https://smartcart-backend-2muc.onrender.com/api/admin/users/6/role?role=ROLE_ADMIN" -H "Authorization: Bearer <admin_token>"
```

**Sample Response (200 OK)**

```
"Role updated successfully"
```

**HTTP Status Codes:**

| HTTP Status   | Meaning                  |
|---------------|--------------------------|
| 200 OK        | Role updated             |
| 403 Forbidden | Caller is not ROLE_ADMIN |

## 7.5 Health Check

**GET** /api/test

Simple reachability/JWT sanity-check endpoint.

**Authorization Required:** Public

**Sample Request (cURL)**

```
curl https://smartcart-backend-2muc.onrender.com/api/test
```

**Sample Response (200 OK)**

```
"JWT Working"
```

**HTTP Status Codes:**

| HTTP Status | Meaning           |
|-------------|-------------------|
| 200 OK      | Service reachable |

## 7.6 Notes & Best Practices — Profile & Admin

- Role values are stored and returned with the ROLE\_ prefix (ROLE\_ADMIN / ROLE\_CUSTOMER), consistent with Spring Security's default GrantedAuthority convention — clients should match against these exact strings, not "ADMIN"/"CUSTOMER".
- There is no endpoint to promote a user to Admin except PUT /api/admin/users/{id}/role, and it is itself Admin-only — the very first admin account must be seeded directly in the database, which is exactly how admin@smartcart.com exists in production today.

## Appendix: Quick-Reference Endpoint Index

---

A condensed, one-line-per-endpoint index of all 49 endpoints, for fast lookup. See the corresponding chapter for full request/response detail.

### A.1 Authentication

| Method | Endpoint           | Access |
|--------|--------------------|--------|
| POST   | /api/auth/register | Public |
| POST   | /api/auth/login    | Public |

### A.2 Products & Categories

| Method | Endpoint                            | Access |
|--------|-------------------------------------|--------|
| POST   | /api/products                       | Admin  |
| GET    | /api/products                       | Public |
| GET    | /api/products/{id}                  | Public |
| PUT    | /api/products/{id}                  | Admin  |
| DELETE | /api/products/{id}                  | Admin  |
| GET    | /api/products/category/{categoryId} | Public |
| GET    | /api/products/search                | Public |
| GET    | /api/products/sort/price-asc        | Public |
| GET    | /api/products/sort/price-desc       | Public |
| GET    | /api/products/paged                 | Public |
| PUT    | /api/products/{id}/stock            | Admin  |
| GET    | /api/products/low-stock             | Admin  |
| POST   | /api/products/upload                | Admin  |
| POST   | /api/categories                     | Admin  |
| GET    | /api/categories                     | Public |
| PUT    | /api/categories/{id}                | Admin  |
| DELETE | /api/categories/{id}                | Admin  |

### A.3 Cart, Wishlist & Reviews

| Method | Endpoint                 | Access   |
|--------|--------------------------|----------|
| POST   | /api/cart                | Customer |
| GET    | /api/cart                | Customer |
| DELETE | /api/cart/{cartId}       | Customer |
| PUT    | /api/cart/{cartId}       | Customer |
| POST   | /api/wishlist            | Customer |
| GET    | /api/wishlist            | Customer |
| DELETE | /api/wishlist/{id}       | Customer |
| POST   | /api/reviews             | Customer |
| GET    | /api/reviews/{productId} | Public   |

### A.4 Orders, Payments & Coupons

| Method | Endpoint                      | Access   |
|--------|-------------------------------|----------|
| POST   | /api/orders/checkout          | Customer |
| GET    | /api/orders                   | Customer |
| GET    | /api/orders/admin             | Admin    |
| PUT    | /api/orders/{id}/status       | Admin    |
| PUT    | /api/orders/cancel/{id}       | Customer |
| GET    | /api/orders/invoice/{orderId} | Customer |
| POST   | /api/payments                 | Customer |
| GET    | /api/payments                 | Customer |
| POST   | /api/coupons                  | Admin    |
| GET    | /api/coupons/validate/{code}  | Public   |

## A.5 Profile, Dashboard, Reports & Admin

| Method | Endpoint                         | Access         |
|--------|----------------------------------|----------------|
| PUT    | /api/profile                     | Customer/Admin |
| PUT    | /api/profile/password            | Customer/Admin |
| GET    | /api/profile/stats               | Customer/Admin |
| GET    | /api/dashboard                   | Admin          |
| GET    | /api/reports/sales               | Admin          |
| GET    | /api/reports/monthly-revenue     | Admin          |
| GET    | /api/reports/product-performance | Admin          |
| GET    | /api/admin/users                 | Admin          |
| DELETE | /api/admin/users/{id}            | Admin          |
| PUT    | /api/admin/users/{id}/role       | Admin          |
| GET    | /api/test                        | Public         |

## A.6 Database Cross-Reference

For traceability, every endpoint above ultimately reads from or writes to the following production tables (see `smartcart_db.sql`): `users`, `categories`, `products`, `cart_items`, `wishlist`, `reviews`, `coupons`, `orders`, `order_items`, `payments` — 10 tables in total, matching the ER diagram in Chapter 5 of the Project Report.